

Data Flow Diagrams & User Stories

Phase: Requirement Analysis

Project: ShopEEZ | Date: June 2026 | Stack: MongoDB · Express · React · Node.js

1. Customer User Stories

US#	Persona	I want to...	So that...	Priority
US-01	Customer	Register with email and password	I can create a personal account	Must Have
US-02	Customer	Log in securely	I can access my cart, orders, profile	Must Have
US-03	Customer	Browse all products	I can discover available items	Must Have
US-04	Customer	Search for specific products	I can quickly find what I need	Must Have
US-05	Customer	Filter by category, price, rating	I can narrow down choices efficiently	Must Have
US-06	Customer	View product details	I can read specs and reviews before buying	Must Have
US-07	Customer	Add products to cart	I can collect items before purchasing	Must Have
US-08	Customer	Cart saved between sessions	I do not lose cart on refresh	Must Have
US-09	Customer	Checkout with shipping address	My order is delivered to the right place	Must Have
US-10	Customer	View order history	I can track past purchases and statuses	Should Have
US-11	Customer	Add to wishlist	I can save favorites without buying immediately	Should Have
US-12	Customer	Submit product reviews	I can share experience with other shoppers	Could Have

2. Admin User Stories

US#	Persona	I want to...	So that...	Priority
US-13	Admin	View dashboard statistics	I can monitor store performance at a glance	Must Have
US-14	Admin	Add new products	I can expand the product catalog	Must Have
US-15	Admin	Edit product details	I can update prices, stock, and images	Must Have
US-16	Admin	Delete products	I can remove discontinued items	Must Have
US-17	Admin	View all customer orders	I can manage fulfillment centrally	Must Have
US-18	Admin	Update order status	I can mark orders as shipped or delivered	Must Have
US-19	Admin	View all registered users	I can monitor the customer base	Should Have
US-20	Admin	See low stock alerts	I can restock before items run out	Should Have

3. Level 0 DFD — Context Diagram

The ShopEEZ system receives inputs from two external entities:

- Customer: Login/Register, Search Products, Add to Cart, Place Order, Review Product
- Admin: Add/Update/Delete Products, Update Order Status, Delete User

The system outputs:

- To Customer: Product Catalog, Order Confirmation, Order Status, Invoice
-

To Admin: Dashboard Stats, Product List, Order List, User List, Revenue Reports

4. Level 1 DFD — Authentication Flow

Customer submits email + password

- Step 1.1: Validate input fields (email format, password length)
- Step 1.2: Query MongoDB users collection by email
- Step 1.3: bcrypt.compare(submitted password, stored hash)
- Step 1.4: If match !' jwt.sign(userId, JWT_SECRET) !' return Bearer token
- Step 1.5: Client stores token in localStorage via Redux authSlice

5. Level 1 DFD — Product Search & Filter

Customer submits: keyword, category, minPrice, maxPrice, rating, sort

- Step 2.1: Parse query params from GET /api/products request
- Step 2.2: Build Mongoose filter object: {name: \$regex, category, price: {\$gte,\$lte}, ratings: {\$gte}}
- Step 2.3: Query MongoDB products collection with filter
- Step 2.4: Apply sort + pagination (skip, limit, sort field)
- Step 2.5: Return JSON: { products[], page, pages, total }

6. Level 1 DFD — Order Placement

Customer submits: cart items + shippingAddress + paymentMethod

- Step 3.1: protect middleware verifies JWT
- Step 3.2: Validate all cart items against Product.stock in MongoDB
- Step 3.3: Create Order document: {user, items, shippingAddress, paymentMethod, orderStatus: processing}
- Step 3.4: Decrement Product.stock for each ordered item
- Step 3.5: Clear customer cart
- Step 3.6: Return 201 Created: { success: true, order }